

Distilling a Hybrid Frontier Pipeline into an Offline Open-Weight Judge for Bengali Hallucination Detection

Team watermelon

IUT Datathon — *Alikbachan* (Bengali Hallucination Detection) Phase 2

shoaibhossain2017@gmail.com

Abstract

We describe our Phase 2 system for the *Alikbachan* Bengali LLM hallucination detection challenge: given a Bengali prompt, an optional supporting passage, and a candidate response, predict whether the response is faithful or hallucinated (binary F1 on the hallucinated class). Our Phase 1 system (private $F1_0 = 0.910$) was a hybrid of a feature-stacked classifier, retrieval against live Bengali Wikipedia, and a frontier-LLM judging layer — components that cannot run in an offline kernel. For Phase 2 we convert that pipeline into a self-contained offline system: the frontier judging layer is distilled into a QLoRA fine-tune of Qwen2.5-7B-Instruct trained on 1,608 labels the Phase 1 system produced, ensembled with a feature stack that now trains on $6\times$ the labeled data Phase 1 had, under a layer of exact deterministic verifiers. We evaluate on the 299 organizer-labeled samples, the only ground truth we hold. These rows receive no gradient updates, but they drive checkpoint selection, threshold calibration and system development, so we report them as a *development set* rather than an untouched hold-out. The shipped judge reaches $F1_0 = 0.711$ there and the data-scaled stack 0.730, combining to 0.732 before deterministic overrides. An aggressive first fine-tune scored only 0.621, below the 0.699 of the un-tuned base model, and the gap is concentrated in closed-book rows. We document label provenance and our Phase 1 reproduction mechanism in full.

1 Task and Setting

Each row of the dataset is a triple: a Bengali prompt (`prompt_bn`), a candidate response (`response_bn`), and optionally a supporting passage (`context`); roughly half the rows are open-book (`context` given) and half are closed-book. A response is labeled 0 (hallucinated) if it is fabricated, unsupported by the given context, or factually incorrect; the metric is binary F1 on class 0. The Phase 1 test set contained 2,516 rows

(1,361 open-book / 1,155 closed-book) spanning factual QA, idiom meanings, grammar and spelling items, multiple-choice questions, and templated arithmetic word problems, across cultural bands from global to Bangladesh-specific facts. Phase 2 is evaluated by the organizers running our notebook on a new held-out fold, offline, under 9 hours on a single P100 or $2\times$ T4, with all model weights under 50 GB.

2 Phase 1 System (Summary)

Our Phase 1 pipeline layered three sources of authority. (1) A stacked classifier (XGBoost + logistic regression, 50/50 blend) over grounding features: multilingual NLI entailment (mDeBERTa-XNLI), extractive-QA answer agreement (XLM-R SQuAD2), multilingual-e5 embedding cosines, and lexical overlap, with decision thresholds tuned separately for open-book and closed-book rows. (2) Retrieval from live Bengali Wikipedia providing evidence for closed-book rows. (3) A frontier-LLM judging layer over all closed-book rows, cross-checked by two open 32B judges (Qwen2.5-32B, Qwen3-32B), with targeted manual verification of disagreements; templated arithmetic was verified exactly by a regex-and-compute solver. This scored 0.910 on the private leaderboard. Three of these components — the frontier judge, live retrieval, and manual verification — cannot run in an offline kernel, which motivates the Phase 2 design.

3 Phase 2 System

3.1 Design principle

Rather than approximating each online component separately, we treat the *outputs* of the Phase 1 pipeline as a teacher signal and distill them into one open-weight student: a judge that answers “is this response correct?” directly. That judge is then ensembled with the Phase 1 feature stack, retrained

Source	Rows	O/1	Provenance
Official samples	299	136/163	organizer labels
Frontier no-ctx	1,123	541/582	frontier LLM ^a
Verified ctx	485	251/234	mixed ^b

Table 1: Label sources. ^aFrontier-LLM judgments of Phase 1 test rows, cross-checked against two open 32B judges; templated-arithmetic subset verified exactly by our solver (135/135). ^bRule-derived labels (grammar templates, regex-verified birth years against passages) plus manually reviewed uncertain rows. The 299 official samples receive **no gradient updates** but are used for model selection and calibration, so we treat them as a development set. Disclosure: a minority of the verified-ctx and frontier labels incorporate human review of Phase 1 test rows. Phase 2 is scored on a disjoint fold, so no human labeling of the evaluated data is involved.

on the enlarged corpus. The offline system is

judge $P(\text{yes}) + \text{stack } P(\text{faithful}) \rightarrow \text{regime blend} \rightarrow \text{regime threshold} \rightarrow \text{overrides}$

with blend weights and thresholds fitted separately for the open- and closed-book regimes, because the two components dominate in opposite regimes (§5).

3.2 Training corpus and label provenance

Table 1 lists the corpus. After a consistency filter (drop rows where both cached 32B judges confidently contradict the stored label — zero rows fired), the training set is 1,608 rows (792 hallucinated / 816 faithful), all drawn from Phase 1 test rows pseudo-labeled by the Phase 1 system. Each row yields two training examples via Bengali and English judge-prompt templates (identical to our Phase 1 judge prompts), giving 3,216 examples.

3.3 Distilled judge

We fine-tune Qwen2.5-7B-Instruct with QLoRA (NF4, $r=16$, $\alpha=32$, dropout 0.1, all linear projections; lr $4 \cdot 10^{-5}$ cosine; max length 1,536; three epochs scheduled with the *first* selected by development-set evaluation; loss on the answer tokens only). The model is trained to answer the Bengali words for yes/no (*hyam/na*) or English Yes/No. At inference we score $P(\text{yes})$ from the first-token logits, summing over the disjoint first-token id sets of the yes and no surface forms (byte-fragment tokens shared between the Bengali words are excluded), and average the Bengali- and English-template probabilities. Thresholds on $P(\text{yes})$ are chosen per side (open-/closed-book) on the 299-sample development set, taking the *median of the*

threshold region within 0.005 of the maximum $F1_0$ rather than the argmax, for robustness to fold shift.

3.4 Deterministic layer

Three exact, content-based overrides run after the judge (increasing authority): (1) **sample match**: a test row whose prompt appears verbatim among the 299 labeled samples is resolved by *exact* normalized-answer equality against the known faithful/hallucinated responses (we removed substring containment, which could mark a contradictory response containing the correct span as faithful); (2) **arithmetic verification**: 14 regex template families parse the digit-normalized prompt, compute the true answer, and label by numeric match (validated 135/135 against derivable Phase 1 truth); (3) **Phase 1 reproduction cache**: a SHA-256 content-keyed (context, prompt, response) lookup of our Phase 1 predictions. On the Phase 1 test file this reproduces our Phase 1 submission exactly (2,516/2,516 rows). It is all-or-nothing: it applies only when at least 99% of the input rows match, so an incidental duplicate on a new fold cannot receive a Phase 1 label. Below that gate it is inert and the distilled system produces every prediction. The notebook logs its hit count.

3.5 Engineering for the offline kernel

The notebook writes a prior-based fallback submission before any GPU work, so a valid output exists at every point; a runtime guard skips the English-template pass if the projected total exceeds 7.5 h. Measured on 2×T4 over the 2,516-row Phase 1 test file, the full pipeline runs well inside the limit, and the organizers’ 5,000-row held-out fold projects to roughly an hour against a 9 h budget. We found during dry runs that the current Kaggle image (PyTorch 2.10, cu128) no longer supports the P100’s compute capability 6.0, so GPU execution on P100 fails for *any* PyTorch model; our notebook catches judge failure and degrades to its prior fallback while still applying every deterministic layer. The notebook writes a baseline submission in its first cell, using only exttt pandas and the required input path, before any asset discovery or import; all remaining work runs inside a single exception boundary and a final cell asserts the output schema, so the notebook completes with a well-formed submission under any downstream failure we tested. Robustness was tested by mutation: shuffled and non-contiguous ids, varying row counts, all three no-context encodings (NaN, [NULL], empty), nu-

System (scored on the 299)	all	ctx	no-ctx
Qwen2.5-7B zero-shot judge	0.699	0.737	0.686
+ QLoRA, 3 epochs, lr 10^{-4}	0.621	0.739	0.564
+ QLoRA, 1 epoch, lr $4 \cdot 10^{-5}$	0.711	0.747	0.704
Stack, trained on 1,608	0.716	0.784	0.690
Stack, trained on 1,907 (CV)	0.730	0.832	0.690
Blend, single global weight	0.733	0.838	0.690
Blend, per-side weights	0.737	0.832	0.698
Phase 1 stack (299-only CV, ref.)	0.770	0.88	0.72
Phase 1 full hybrid (private LB)	0.910	—	—

Table 2: $F1_0$ on the 299 organizer labels, measured *extbf* before the deterministic override layer, i.e. for the base ensemble only. The judge receives no gradient updates on these rows; the 1,907-row stack row is 5-fold cross-validated over the 299 so no row is scored by a model that saw it. Phase 1 rows are reference points, measured differently (CV on the 299 alone, and the private leaderboard).

meric responses, quoted newlines, and reordered or missing columns.

4 Results

Table 2 isolates what survives the move offline. The distilled judge recovers the frontier layer’s behaviour on open-book rows (0.739), where the evidence needed to decide is present in the prompt, but only 0.564 on closed-book rows, where deciding requires knowing the fact. This is the expected failure mode of distillation: judgments transfer, knowledge does not. Scaling the stack’s training data from the 299 organizer rows to 1,907 (the Phase 1 pseudo-labels plus the 299) lifts open-book $F1_0$ to 0.832; the closed-book side is unmoved at 0.690, confirming the same bottleneck from the other direction. Blending contributes little once the stack has the extra data (the fitted judge weight is 0.0–0.3 depending on the stack’s training set), because the two components fail on the same rows. Per question type, the blend scores 0.79 on dates, 0.79 on arithmetic, 0.78 on multiple choice, 0.77 on idiom meanings, 0.72 on *who* questions, and 0.69 on the residual *other* category, which dominates by count ($n = 166$) and is where closed-book world knowledge matters most.

5 Analysis and Limitations

What does not distil. Our central negative result is that the closed-book gap is not a training-recipe problem. Two independent interventions — distilling 1,608 frontier judgments into the student,

and giving the feature stack $6 \times$ more training data — both leave closed-book $F1_0$ at 0.69 *on this development set*, while open-book $F1_0$ moves from 0.784 to 0.832. A sweep over 13 stack configurations (tree depth 3–6, 300–1000 estimators, two blend weights) reproduces closed-book $F1_0$ of exactly 0.6899 in twelve of them, so the ceiling is not model capacity either. The Phase 1 teacher answered closed-book items using world knowledge obtained from live retrieval and web verification; a 7B student fine-tuned on the *outputs* of that process learns the decision style, not the facts. Where the evidence needed to decide is in the input, distillation works; where it is in the world, it does not.

Fine-tuning can destroy what it cannot replace.

Table 2 contains our most useful diagnostic, though we note up front that a 299-row development set and a single 7B student cannot establish general claims about distillation. Under the recipes we evaluated: the *zero-shot* base model already scores 0.686 on closed-book rows. Our first fine-tune (lr 10^{-4} , 3 epochs, training loss 0.002) drove that *down* to 0.564 — distillation on judgments the student cannot justify from the input does not merely fail to add knowledge, it overwrites calibrated uncertainty with confident imitation. Re-training at lr $4 \cdot 10^{-5}$ with checkpoint selection by per-epoch holdout evaluation recovers 0.704, and the epoch trajectory is monotone downward thereafter (0.704, 0.638, 0.620), so the useful signal is exhausted within one pass. Open-book rows show no such collapse (0.737 zero-shot to 0.747), consistent with grounded verification being genuinely learnable from these labels while closed-book factuality is not.

Per-side blending. Because the two components fail in opposite regimes — the stack is far stronger with a passage (0.832 vs. 0.747), the judge slightly stronger without one (0.698 vs. 0.690) — a single global blend weight averages two different optima. Fitting the weight separately per evidence regime on the holdout selects essentially the stack for open-book rows and the judge for closed-book rows, and is the configuration we ship.

Other limits. Arithmetic verification was exact on the recognised Phase 1 template families and is not evidence about unseen templates; it fires only on a full structural match and is skipped on any parse error, so its risk is a missed override

rather than a confident wrong one. unparsed math rows fall back to the ensemble, which has learned a strong hallucinated prior for closed-book arithmetic. Pseudo-label noise bounds the student from above; we mitigate with cross-judge consistency filtering and exact verification where possible, and teacher accuracy was 0.95 on the 41 closed-book rows whose ground truth happened to be derivable, a selected subset that need not represent the teacher overall. Finally, all thresholds are calibrated on 299 rows, so we choose flat-region medians rather than argmax values to reduce variance under fold shift.

Engineering findings for offline kernels. Several environment facts cost us dry-run cycles and are worth recording. The current Kaggle image (PyTorch 2.10, cu128) no longer supports the P100’s compute capability 6.0, so GPU PyTorch cannot run on that accelerator at all. `bitsandbytes` is absent from the image, ruling out 4-bit inference without a vendored wheel. `peft` 0.19 raises on a `torchao` version check when attaching an adapter, so we merge LoRA weights directly ($W += (\alpha/r)BA$) and drop the dependency. Locating the base model by globbing for `model*.safetensors` silently finds the wrong network once encoder mirrors are also attached, so we match on `config.json` architecture instead. Finally, sorting prompts by length for padding efficiency concentrates the longest sequences in the final batches, which OOMs a fixed batch size partway through a run; we batch to a token budget and recursively halve any batch that still fails. Because the notebook writes a valid submission before any GPU work and treats every model stage as optional, each of these failures degraded rather than disqualified the run.

6 Compliance and Reproducibility

All inference-time models are open-weight (Qwen2.5-7B-Instruct base + our LoRA adapter; both attached as Kaggle datasets/models), no internet access is required, and total weights are ≈ 18 GB (15.2 GB base, 0.15 GB adapter, 2.8 GB encoders). Fine-tuning on the labeled sample set and own models are explicitly permitted by the competition rules; we additionally disclose all label provenance in Table 1 and the reproduction-cache mechanism in §3.4. The README lists the exact environment and artifacts.

Acknowledgments

We thank the organizers for the dataset and for the Phase 2 infrastructure.

References

- [1] A. Yang et al. Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS*, 2023.
- [3] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [4] G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. *arXiv:1503.02531*, 2015.
- [5] L. Wang et al. Multilingual E5 Text Embeddings. *arXiv:2402.05672*, 2024.
- [6] T. Chen, C. Guestrin. XGBoost: A Scalable Tree Boosting System. *KDD*, 2016.
- [7] A. Conneau et al. XNLI: Evaluating Cross-lingual Sentence Representations. *EMNLP*, 2018.
- [8] A. Conneau et al. Unsupervised Cross-lingual Representation Learning at Scale. *ACL*, 2020.
- [9] Y. Zhang et al. Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *arXiv:2309.01219*, 2023.